

CRYPTOGRAPHY

& INFORMATION SYSTEM SECURITY



Modul praktikum: Hashing

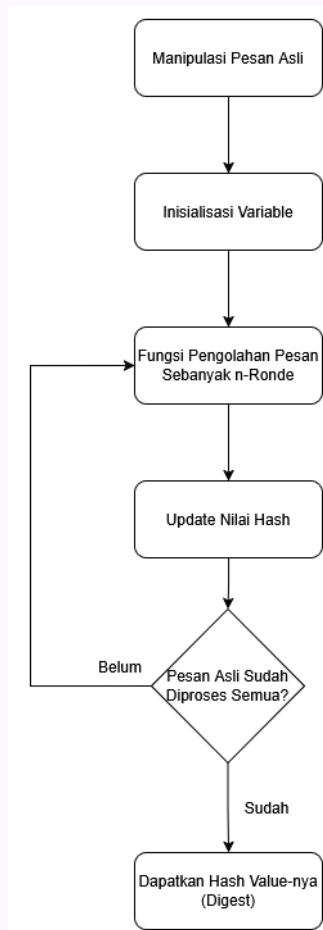
Daftar Isi

Daftar Isi.....	2
Overview Hashing.....	3
SHA.....	3
Overview.....	3
SHA-1.....	3
SHA-3.....	4
Tools SHA.....	7
Perbandingan dengan MAC.....	7
Tugas.....	7
Daftar Pustaka.....	8
Tim Asisten Praktikum.....	8

Overview Hashing

Hash Function adalah fungsi yang menerima pesan dengan panjang variabel (M) sebagai input dan menghasilkan output dengan panjang tetap $h = H(M)$ (Stallings, 2023). Menurut Stallings (2023), jenis fungsi hash yang dibutuhkan dalam kriptografi disebut sebagai fungsi hash kriptografi.

Sederhananya, proses Hashing terdiri dari beberapa langkah sebagai berikut:



Penjelasan:

1. Pada pemrosesan pesan pada SHA, kita **wajib mengikuti aturan**. Contohnya, pada **SHA-1** kita **wajib** memiliki **pesan asli** sepanjang **512-bit** sebelum diproses di fungsi nya. Lalu, bagaimana kalau pesan kita hanya 'abc' (3 byte=24 bit)? **Lakukan Padding**.
2. Pada SHA-1 maupun 3, kita perlu melakukan tahapan **inisialisasi**. Pada SHA-1, terdapat variabel **Hash Buffer (H)**, **Konstanta (K)**, dan **Register (A sampai F)**.
3. Setelah pesan asli sudah sesuai **aturan** dan **variabel** sudah diinisialisasikan, **Fungsi Hashing** sudah dapat **dilakukan**. Fungsi hashing akan disisipi **Pesan Asli** yang sudah diproses (dan dipotong-potong sesuai ketentuan, tidak mentah **512-bit** misal), serta dijalankan berulang kali. Contohnya pada **SHA-3** dilakukan sebanyak **24 kali** (istilahnya ronde/putaran).
4. Setelah **fungsi hashing selesai** (dilakukan sebanyak ronde yang ditentukan), pastinya **memiliki output**. Output tersebut akan **mengupdate Hash Value/Buffer** yang sudah diinisialisasikan pada tahap 2.
5. Apakah pesan asli **sudah semuanya** memiliki **giliran untuk "dipakai"** pada **fungsi hashing**? Jika masih ada yang **belum**, misal pesan asli adalah **700 bit SHA-1**, maka dilakukan **dua kali fungsi hashing, blok pertama: 512-bit & blok kedua: 188-bit**.
6. Jika pesan asli semuanya sudah diproses/dapat giliran, Hash Value sudah bisa didapatkan.

SHA

Overview

SHA adalah fungsi hash satu arah yang dibuat oleh NIST dan digunakan bersama DSS atau Digital Signature Standard (Munir, 2019).

SHA-1

Salah satu varian SHA adalah SHA-1. Secara garis besar, algoritma SHA-1 menurut Munir adalah sebagai berikut.

1. Penambahan padding bits.
2. Penambahan nilai panjang pesan semula.
3. Inisialisasi MD buffer.
4. Pengolahan pesan dalam block berukuran 512 bit.

Algoritma 1 menunjukkan bagaimana implementasi SHA-1 tersebut.

[Psst, bingung? Boleh lihat PPT nih!](#)

Algoritma 1.1 Contoh Implementasi SHA-1 C++

```
#include <iostream>
#include <vector>
#include <string>
#include <stdint>
#include <stdio>

using namespace std;

// --- Helper untuk Operasi Bitwise ---
#define ROTL(bits, word) (((word) << (bits)) | ((word) >> (32 - (bits))))

struct SHA1_State {
    uint32_t H[5];
};

// LANGKAH 4: Pengolahan pesan dalam blok 512 bit (The Transform Engine)
void prosesBlok(SHA1_State &state, const uint8_t blok[64]) {
    uint32_t w[80];
    for (int i = 0; i < 16; i++) {
        w[i] = (blok[i*4] << 24) | (blok[i*4+1] << 16) | (blok[i*4+2] << 8) |
        (blok[i*4+3]);
    }
    for (int i = 16; i < 80; i++) {
        w[i] = ROTL(1, w[i-3] ^ w[i-8] ^ w[i-14] ^ w[i-16]);
    }

    uint32_t a = state.H[0], b = state.H[1], c = state.H[2], d = state.H[3], e =
state.H[4];

    for (int i = 0; i < 80; i++) {
        uint32_t f, k;
        if (i < 20) { f = (b & c) | ((~b) & d); k = 0x5A827999; }
        else if (i < 40) { f = b ^ c ^ d; k = 0x6ED9EBA1; }
        else if (i < 60) { f = (b & c) | (b & d) | (c & d); k = 0x8F1BBCDC; }
        else { f = b ^ c ^ d; k = 0xCA62C1D6; }

        uint32_t temp = ROTL(5, a) + f + e + k + w[i];
        e = d; d = c; c = ROTL(30, b); b = a; a = temp;
    }

    state.H[0] += a; state.H[1] += b; state.H[2] += c; state.H[3] += d; state.H[4] += e;
}
```

```

}

string hitungSHA1(string pesan) {
    // LANGKAH 3: Inisialisasi MD Buffer
    SHA1_State state;
    state.H[0] = 0x67452301;
    state.H[1] = 0xEFCDAB89;
    state.H[2] = 0x98BADCFE;
    state.H[3] = 0x10325476;
    state.H[4] = 0xC3D2E1F0;

    // Persiapan data (mengubah string ke byte array)
    vector<uint8_t> data(pesan.begin(), pesan.end());
    uint64_t bitLength = data.size() * 8;

    // LANGKAH 1: Penambahan Padding Bits
    // Tambahkan bit '1' (0x80 dalam byte)
    data.push_back(0x80);
    // Tambahkan bit '0' sampai panjangnya bersisa 64 bit dari kelipatan 512
    while ((data.size() % 64) != 56) {
        data.push_back(0x00);
    }

    // LANGKAH 2: Penambahan Nilai Panjang Pesan Semula (64-bit Big Endian)
    for (int i = 7; i >= 0; i--) {
        data.push_back((uint8_t)(bitLength >> (i * 8)));
    }

    // LANGKAH 4 (Lanjutan): Pengolahan pesan dalam blok berukuran 512 bit
    for (size_t i = 0; i < data.size(); i += 64) {
        prosesBlok(state, &data[i]);
    }

    // Konversi hasil ke format Hex
    char hexOutput[41];
    for (int i = 0; i < 5; i++) {
        sprintf(hexOutput + (i * 8), "%08x", state.H[i]);
    }
    return string(hexOutput);
}

int main() {
    string input = "abc";
    cout << "Input : " << input << endl;
    cout << "SHA-1 : " << hitungSHA1(input) << endl;
    return 0;
}

```

Algoritma 1.2 Contoh Implementasi SHA-1 Python

```

def ROTL(bits, word):
    return ((word << bits) | (word >> (32 - bits))) & 0xFFFFFFFF

def prosesBlok(H, blok):
    w = [0] * 80

    # 1. Bangun 16 Word pertama (dari kepingan 8-bit menjadi 32-bit Big-Endian)
    for i in range(16):
        w[i] = (blok[i*4] << 24) | (blok[i*4+1] << 16) | (blok[i*4+2] << 8) | blok[i*4+3]

    # 2. Ekspansi menjadi 80 Word
    for i in range(16, 80):
        w[i] = ROTL(1, w[i-3] ^ w[i-8] ^ w[i-14] ^ w[i-16])

```

```

# 3. Inisialisasi variabel sementara
a, b, c, d, e = H[0], H[1], H[2], H[3], H[4]

# 4. Siklus 80 Putaran
for i in range(80):
    if i < 20:
        f = (b & c) | ((~b & 0xFFFFFFFF) & d)
        k = 0x5A827999
    elif i < 40:
        f = b ^ c ^ d
        k = 0x6ED9EBA1
    elif i < 60:
        f = (b & c) | (b & d) | (c & d)
        k = 0x8F1BBCDC
    else:
        f = b ^ c ^ d
        k = 0xCA62C1D6

    # Hitung TEMP dan gunakan & 0xFFFFFFFF untuk meniru Modulo 2^32
    temp = (ROTL(5, a) + f + e + k + w[i]) & 0xFFFFFFFF

    # Geser posisi Register
    e = d
    d = c
    c = ROTL(30, b)
    b = a
    a = temp

# 5. Update Nilai Hash Utama (selalu pastikan terpotong di 32-bit)
H[0] = (H[0] + a) & 0xFFFFFFFF
H[1] = (H[1] + b) & 0xFFFFFFFF
H[2] = (H[2] + c) & 0xFFFFFFFF
H[3] = (H[3] + d) & 0xFFFFFFFF
H[4] = (H[4] + e) & 0xFFFFFFFF

def hitungSHA1(pesan):
    # LANGKAH 3: Inisialisasi MD Buffer (Menggunakan List/Array biasa)
    H = [0x67452301, 0xEFCDAB89, 0x98BADCFE, 0x10325476, 0xC3D2E1F0]

    # Persiapan data: bytearray mirip dengan vector<uint8_t> di C++
    data = bytearray(pesan.encode('utf-8'))
    bitLength = len(data) * 8

    # LANGKAH 1: Penambahan Padding Bits
    data.append(0x80)
    while (len(data) % 64) != 56:
        data.append(0x00)

    # LANGKAH 2: Penambahan Nilai Panjang Pesan Semula (64-bit Big Endian)
    for i in range(7, -1, -1):
        # & 0xFF memastikan kita hanya mengambil 8-bit paling kanan (meniru uint8_t)
        data.append((bitLength >> (i * 8)) & 0xFF)

    # LANGKAH 4 (Lanjutan): Pengolahan pesan per blok 512 bit (64 Byte)
    for i in range(0, len(data), 64):
        prosesBlok(H, data[i : i+64])

    # Konversi array H menjadi format string Hexadesimal bersambung
    return ''.join(f'{x:08x}' for x in H)

if __name__ == "__main__":
    input_text = "abc"
    print(f"Input : {input_text}")
    print(f"SHA-1 : {hitungSHA1(input_text)}")

```

SHA-3

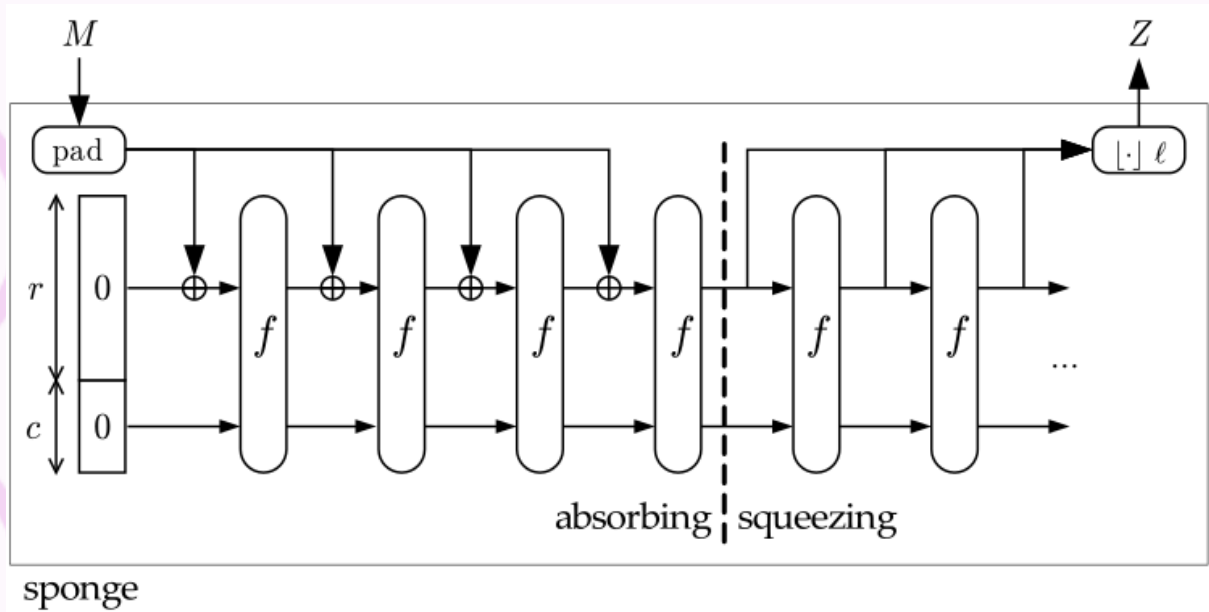
Perbedaan SHA-1 dan SHA-3(Keccak) ditunjukkan pada Tabel 1.

Tabel 1. Perbandingan SHA-1 dan SHA-3

Kriteria	SHA-1	SHA-3
Arsitektur dasar	Merkle-Damgard (Iterative Compression)	Sponge Construction (Absord & Squeeze)
Logika Operasi	Matematis (Penjumlahan Modular, XOR, AND, OR, Rotasi)	Logika Bitwise Murni (XOR, AND, NOT) dan Permutasi Posisi
Struktur Internal	Array linear (5 register 32-bit: A, B, C, D, E)	Matriks 3D/State 5x5x64 (Total 1600 bit untuk versi standar)
Ukuran Block	Tetap 512 bit	Dinamis (Tergantung nilai Rate r)
Panjang Output	Tetap 160 bit	Fleksibel (224, 256, 384, 512 bit, atau sesuai kebutuhan pada SHAKE)
Tahapan Utama	1. 2. 3. Padding Inisialisasi 80 Round Transformasi	1. 2. 3. Padding Absorbing (Menyerap daa ke state) Squeezing (Memeras hash dari State)
Keamanan (Matematis)	Rentan terhadap serangan Collision	Tahan terhadap serangan Length Extension

Sponge Construction adalah konstruksi iteratif sederhana untuk membangun fungsi F dengan input panjang variable dan panjang output sembarang berdasarkan permutasi/transformasi panjang tetap f yang beroperasi pada sejumlah bit tetap b (state) (Bertoni dkk., t.t.-b).

Gambar 1. Konstruksi Spons
Sumber: Bertoni dkk, t.t.-b



Bertoni dkk. (t.t.-b) menjelaskan bahwa konstruksi ini **dimulai** dengan string (input) yang diberikan padding dengan aturan reversible padding (10^*1 , dimulai 1 ditutup 1) dan dapat dipotong ke dalam r bits block. Kemudian State (b) bit dari state diinisiasi menjadi 0 dan proses sponge construction ini dibagi menjadi dua fase:

1. Fase penyerapan (absorbing). Untuk setiap blok masukan berukuran r bits, XOR-kan dengan r bits pertama dari state S , lalu hasilnya dimasukkan ke dalam fungsi permutasi f untuk menghasilkan state baru (b). Ketika semua blok masukan sudah diproses, fase beralih ke pemerasan.
2. Fase pemerasan (squeezing). Inisiasi Z dengan string kosong. Selagi panjang Z belum sama dengan d , r bits pertama dari state S ditambahkan (append) ke Z . Jika panjang Z masih belum sama dengan d , masukkan ke dalam fungsi permutasi f menghasilkan state baru S .

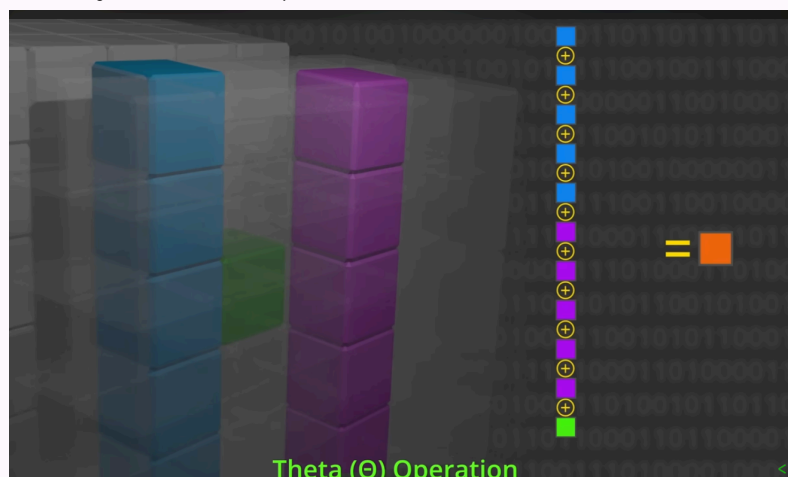
Catatan:

1. b = State ($b = r+c$), terdiri dari 1600 bit
2. c = Capacity (Rule of Thumb: $c = 2 * d$) [Hanya dimanipulasi pada fungsi keccak]
3. r = Rate (Sejumlah bit yang akan dimainkan dengan data input atau output)
4. d = Panjang hash (output)
5. f = fungsi keccak, terdiri dari $(\theta, \rho, \pi, \chi, \iota)$ [Theta, Rho, Pi, Chi, Iota], sebanyak 24 putaran.

[Psst, bingung juga? Sini liat ppt!](#)

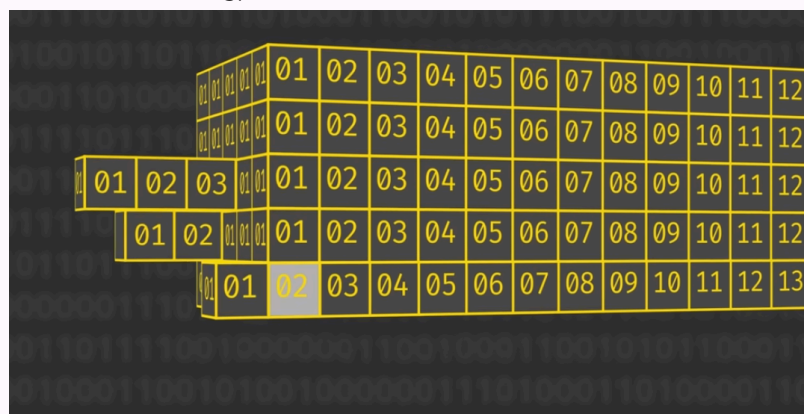
Fungsi Keccak-f

1. Theta (Efek Penyebaran Paritas)



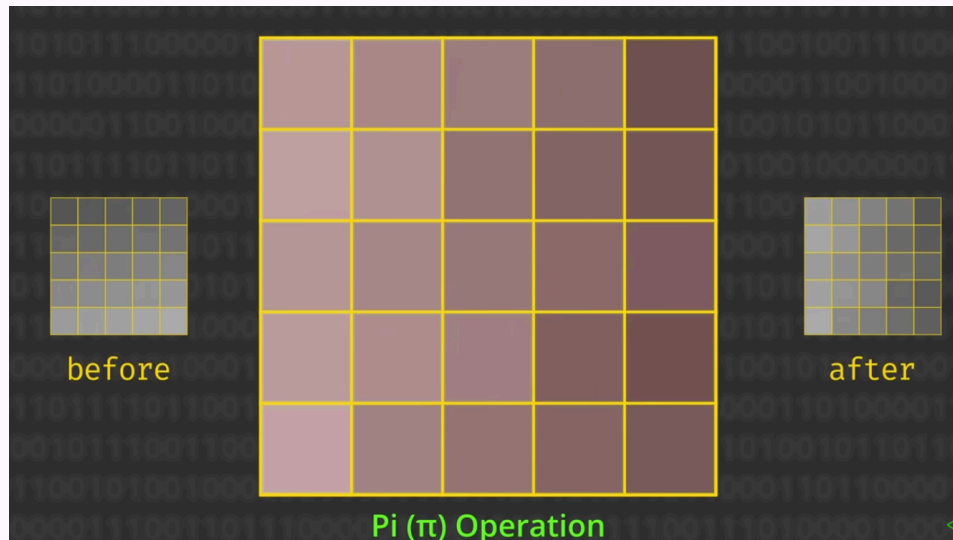
- a. **Theta (Θ) Operation**
- b. Setiap bit di titik (x,y,z) akan di-XOR-kan dengan jumlah paritas (XOR sum) dari kolom di kirinya $(x-1)$ dan kolom di kanannya $(x+1)$, tapi kolom kanannya digeser 1 bit ke belakang $(z-1)$.

2. Rho (Rotasi Internal Lorong)



- a.
- b. Setiap lorong dari total 25 lorong akan di-Left Rotate ($\ll$) dengan jumlah putaran yang sudah ditetapkan secara matematis (bukan acak). Lorong (0,0) tidak diputar (0 bit).

3. Pi (Permutasi Posisi Lorong)



- a.
 - b. Lorong yang berada di posisi (x,y) akan dipindahkan secara utuh ke posisi koordinat baru yaitu $(y, (2x+3y) \pmod{5})$.
4. Chi (Operasi Non-Linear)



- a.
 - b. Mengubah nilai bit berdasarkan gerbang logika AND dan NOT dari tetangganya.
Rumus Mutlak: $A[x] = A[x] \oplus ((\neg A[x+1]) \wedge A[x+2])$
5. Iota (Konstanta Asimetri)
- a. Komputer men-XOR-kan sebuah Konstanta Putaran (*Round Constant/RC*) berukuran 64-bit HANYA ke satu lorong, yaitu lorong pusat di koordinat $(0,0)$. Nilai RC berbeda-beda di tiap putaran (0 hingga 23).

Dengan demikian pseudocode-nya menjadi seperti pada Algoritma 2 (Bertoni dkk., t.t.-a).

Algoritma 2. *Konstruksi Spons*
Sumber: Bertoni dkk, t.t.-a

```

Keccak[r,c](Mbytes || Mbits) {
  # Padding
  d = 2^|Mbits| + sum for i=0..|Mbits|-1 of 2^i*Mbits[i]
  P = Mbytes || d || 0x00 || ... || 0x00
  P = P xor (0x00 || ... || 0x00 || 0x80)

  # Initialization
  S[x,y] = 0,                                     for (x,y) in (0..4,0..4)
A
  # Absorbing phase

```

```

for each block Pi in P
    S[x,y] = S[x,y] xor Pi[x+5*y],          for (x,y) such that x+5*y < r/w
    S = Keccak-f[r+c](S)

# Squeezing phase
Z = empty string
while output is requested
    Z = Z || S[x,y],                      for (x,y) such that x+5*y < r/w
    S = Keccak-f[r+c](S)

return Z
}

```

Untuk implementasi keseluruhannya, Bertoni dkk (t.t.-a) menyediakan library yang sudah disusun readable. Berikut contoh penggunaannya.

1. Unduh library [di sini](#) atau salin seluruh code [dari sini](#).
2. Struktur project menjadi seperti berikut.

```

(project name)/
├─ Keccak-readable-and-compact.c
└─ main.cpp or main.py

```
3. Salin Algoritma 3 ke dalam main.cpp atau main.py

Algoritma 3.1 Konstruksi Spons C++

```

#include <iostream>
#include <iomanip>
#include <string>

extern "C" {
    #include "Keccak-readable-and-compact.c"
}

int main() {
    std::string pesan = "FufuFafa";
    unsigned char hash[32]; // Buffer untuk 256-bit

    // Memanggil fungsi Keccak dari file yang di-include
    // Parameter: rate=1088, capacity=512, input, len, suffix=0x06 (SHA-3), output,
    outLen=32
    Keccak(1088, 512, (const unsigned char*)pesan.c_str(), pesan.length(), 0x06, hash, 32);

    std::cout << "Input : " << pesan << std::endl;
    std::cout << "SHA3-256: ";
    for(int i = 0; i < 32; i++) {
        std::cout << std::hex << std::setw(2) << std::setfill('0') << (int)hash[i];
    }
    std::cout << std::dec << std::endl;

    return 0;
}

```

Algoritma 3.2 Konstruksi Spons Python

```

import ctypes
import os

# 1. Muat (Load) Shared Library C yang sudah kita kompilasi tadi
# Gunakan 'keccak.so' jika Anda di Linux/Mac
lib_path = os.path.abspath("keccak.dll")
keccak_lib = ctypes.CDLL(lib_path)

# 2. Definisikan tipe argumen (Signature) agar Python tidak salah mengirim tipe memori ke C

```

```

keccak_lib.Keccak.argtypes = [
    ctypes.c_uint,          # rate
    ctypes.c_uint,          # capacity
    ctypes.c_char_p,        # input (pointer ke string/bytes)
    ctypes.c_ulonglong,     # inputByteLen
    ctypes.c_ubyte,         # delimitedSuffix
    ctypes.POINTER(ctypes.c_ubyte), # output (pointer ke array byte)
    ctypes.c_ulonglong      # outputByteLen
]

def hitung_SHA3_256_via_C(pesan):
    # Ubah string Python menjadi bytes (C-string)
    input_bytes = pesan.encode('utf-8')
    input_len = len(input_bytes)

    # Siapkan "ruang kosong" di memori untuk diisi oleh fungsi C
    # Karena SHA3-256 butuh 32 byte, kita buat array C berukuran 32
    output_buffer = (ctypes.c_ubyte * 32)()

    # 3. Panggil fungsi C-nya!
    keccak_lib.Keccak(
        1088,          # rate
        512,           # capacity
        input_bytes,   # data input
        input_len,     # panjang input
        0x06,          # suffix untuk SHA-3
        output_buffer, # tempat menaruh hasil
        32             # panjang output yang diminta
    )

    # 4. Ubah array C yang sudah terisi kembali menjadi string Hex Python
    return bytes(output_buffer).hex()

if __name__ == "__main__":
    teks = "FufuFafa"
    hasil_hash = hitung_SHA3_256_via_C(teks)

    print(f"Input    : {teks}")
    print(f"SHA3-256: {hasil_hash}")

```

4. Cara Compile:

- a. **UNTUK C++:** Compile seperti biasa
- b. **Untuk Python:**
 - i. **Windows:** Compile library dengan: `gcc -shared -o keccak.dll Keccak-readable-and-compact.c`
 - ii. **Linux/Mac:** Compile library dengan: `gcc -shared -o keccak.so -fPIC Keccak-readable-and-compact.c`

Tools SHA

Selain mengimplementasikan algoritma *hashing* secara manual (*from scratch*) seperti pada bagian sebelumnya, pada praktiknya di industri dan pengembangan perangkat lunak, kita menggunakan *tools* atau *library* standar yang sudah teruji efisiensi dan keamanannya. Berikut adalah beberapa *tools* yang umum digunakan dan dapat dieksplorasi pada praktikum ini:

1. Command Line Interface (CLI) Bawaan Sistem Operasi

Sebagian besar sistem operasi modern sudah dilengkapi dengan utilitas bawaan untuk melakukan

hashing pada sebuah dokumen atau *file* (sangat berguna untuk mengecek integritas data/mendeteksi perubahan *file*).

- **Windows (PowerShell):** Menggunakan perintah `Get-FileHash`. Contoh: `Get-FileHash -Path dokumen.txt -Algorithm SHA256`
- **Windows (CMD):** Menggunakan utilitas `certutil`. Contoh: `certutil -hashfile dokumen.txt SHA256`
- **Linux / macOS:** Menggunakan perintah *built-in* dari keluarga `shasum`. Contoh: `sha1sum dokumen.txt` atau `sha256sum dokumen.txt`

2. Library Bawaan Bahasa Pemrograman (Standard Library)

Saat mengembangkan aplikasi web atau sistem informasi, kita cukup memanggil *library* bawaan bahasa pemrograman tanpa harus menulis ulang rumusnya.

- **Python:** Menggunakan modul bawaan `hashlib`. Modul ini mendukung berbagai varian (SHA-1, SHA-256, SHA-512, hingga keluarga SHA-3). Contoh implementasi:

Algoritma 4 Contoh `hashlib` Python

```
import hashlib
pesan = b"FuFuFaFa"
hasil_hash = hashlib.sha3_256(pesan).hexdigest()
print(hasil_hash)
```

- **C++:** Menggunakan *library* eksternal standar kriptografi seperti **OpenSSL** (`<openssl/sha.h>`) atau **Crypto++**.

Perbandingan dengan MAC

MAC adalah fungsi hash satu-arah yang menggunakan kunci rahasia dalam pembangkitan nilai hash (Munir, 2019). Maka dari itu, MAC sering disebut juga *keyed hash function*.

Algoritmanya dapat dibuat dengan dua pendekatan, yaitu algoritma berbasis block cipher dan berbasis fungsi hash satu-arah. Contohnya, algoritma berbasis fungsi hash satu-arah dengan pendekatan SHA-1. Misalnya, Alice dan Bob berbagi kunci rahasia (K) untuk bertukar pesan (M). Alice meng-*concat* pesan M dengan K, lalu menghitung nilai hash dari penyambungan itu dengan fungsi hash H. Hasil dari fungsi itulah yang menjadi nilai MAC dari pesan tersebut. Nilai tersebut dikirimkan kepada Bob agar Bob dapat memverifikasi pesan yang didapatnya.

Tabel 2. Perbandingan Hash dan MAC

Kriteria	Hash	MAC
Input utama	Message	Message dan Kunci rahasia
Prinsip dasar	Fungsi satu arah tanpa kunci	Fungsi satu arah dengan kunci simetris
Autentikasi	Tidak ada (hanya fingerprint)	Ada (Dibuktikan dengan kunci rahasia)
Keamanan	Melindungi perubahan tidak sengaja	Melindungi pemalsuan yang disengaja

Tugas

1. OTW!!!!

Deadline: DD/MM/YY HH:MM

Link Pengumpulan



Daftar Pustaka

Bertoni, G., Daemen, J., Hoffert, S., Peeters, M., Van Assche, G., & Van Keer, R. (t.t.-a). Keccak specifications summary. *Team Keccak*. Diambil https://keccak.team/keccak_specs_summary.html

Bertoni, G., Daemen, J., Hoffert, S., Peeters, M., Van Assche, G., & Van Keer, R. (t.t.-b). The sponge and duplex constructions. *Team Keccak*. Diambil https://keccak.team/sponge_duplex.html

Munir, R. (2019). *Kriptografi*. Informatika.

Stallings, W. (2023). *Cryptography and network security: Principles and practice* (Eighth edition, global edition). Pearson.

Tim Asisten Praktikum

1. Franklin
2. Lyan (PIC Praktikum)
3. Ichsan
4. **Alvin (PIC Praktikum)**